

Document Number: P2397R0

Date: 2021-06-15

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [April 14th, 2021](#)
- [April 28th, 2021](#)
- [May 12th, 2021](#)
- [May 26th, 2021](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)

April 14th, 2021

Draft agenda:

- [P2295R2: Correct UTF-8 handling during phase 1 of translation](#)
- [P2348R0: Whitespaces Wording Revamp](#)

Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

Meeting summary:

- PBrett introduced the agenda.
- [P2295R2: Correct UTF-8 handling during phase 1 of translation](#)
 - Corentin introduced:
 - This is a proposal to require that UTF-8 be one of the set of otherwise implementation-defined source file encodings.
 - With regard to ill-formed code unit sequences, there is no such thing; the source code is either valid UTF-8 or it is not UTF-8.
 - Gcc does not validate its presumed UTF-8 input.
 - With regard to BOMs, the proposal does not impose any requirements other than that a BOM present in a UTF-8 source file be ignored for the purposes of lexing.
 - An implementation may use the presence or non-presence of a BOM as part of its source file encoding determination.
 - The proposed wording will require updates for changes that will presumably be adopted from Jens' [P2314: Character sets and encodings](#).
 - This proposal follows Beman Dawes' earlier proposal, [N3463: Portable Program Source Files](#).
 - At present, the C++ standard has no requirement for a portable source file.
 - Tom stated that gcc will perform UTF-8 validation if both `-finput-charset=utf-8` and `-fexec-charset=utf-8` are specified.
 - *[Editor's note: Tom was wrong (and since Tom is also the editor, he can be blunt like that); gcc only validates UTF-8 for string literals, and then only if `-fexec-charset=<encoding>` is specified.]*
 - Jens noted a capitalization issue in the wording; the sentence following the added note in [lex.phases]p1 has a capitalized "The" following a ";".
 - Jens asked why the note added to [lex.phases]p1 is just a note; the preceding prose provides a definition, but does not impose any requirements.
 - PBrett responded that, if an invalid sequence is present, then there is no sequence of Unicode scalar values.

- PBrett asked if moving the note after the following sentence would resolve the concern.
- Jens replied that it would not; that would define a UTF-8 source file and state that a well-formed UTF-8 source file must be accepted, but would impose no requirements on an ill-formed UTF-8 source file.
- PBrett acknowledged that further wording work is needed.
- Jens observed, and noted that the paper discusses, that implementations can accept source files that approximate UTF-8.
- Hubert noted that a normative statement is needed to state that it is implementation-defined how a requirement for UTF-8 source files is specified.
- PBindels suggested placing a requirement for well-formed input with the character set definitions.
- Jens indicated no objection to clarification, but that he would like to see the ISO 10646 definition of "well-formed".
- Steve observed that the note is stating that invalid UTF-8 sequences cannot happen in a well-formed UTF-8 source file.
- Jens responded that there is a normative difference between something that cannot happen and something that is ill-formed; the latter requires a diagnostic.
- Hubert asserted that the wording needs to establish intent; a sequence of bytes may happen to be well-formed UTF-8, but the wording needs to ensure that the bytes were intended to be interpreted as UTF-8.
- PBindels summarized; we need to state there is an implementation-defined way to specify that a source file is to be interpreted as UTF-8.
- Jens agreed.
- JenaHeyd agreed from chat, "Yes, Hubert's definition is correct. You have to make it so the implementation has a way to mark/identify a source file as UTF-8, and then you can impose these requirements."
- Corentin stated the intent; that the compiler determine the source encoding in an implementation-defined way, but that a source file that does not decode successfully is diagnosed as ill-formed.
- Tom suggested specifying that the file must decode successfully as opposed to being well-formed.
- PBrett stated that a branch is needed in translation phase 1 to distinguish the cases where the source file is encoded as UTF-8 vs some other encoding.
- Zach suggested that a definition for a UTF-8 source file is unnecessary.
- PBindels expressed concern that there may be a conflict between use of a BOM and a truly portable source file.
- PBrett responded that the goal is that, if a source file is UTF-8 encoded, that there is a way to direct an implementation to process it as such.
- Jens acknowledged and added that an implementation could require use of a command line option to opt-in to UTF-8 encoded source files; that implies that the source file is not automatically portable, but is the best we can do.

- Tom agreed and stated that the only way we could do better is to require a BOM everywhere and nobody wants that.
- Zach noted that the only statement made regarding a BOM is that it can be ignored; presumably after encoding determination is complete so that the BOM doesn't interfere with translation phase 2.
- Hubert noted that, once the encoding is determined to be UTF-8, a BOM is portably ignored.
- PBrett encouraged assumption of non-hostile implementations; no implementation is going to require a BOM in order for a UTF-8 encoded source file to be processed as such.
- Several relevant comments were made from chat:
 - Steve: "We want portable source code. If anyone requires a BOM, then portable source code needs one."
 - JeanHeyd: "If you put in a BOM and use -fexec-charset=SHIFT-JIS, the implementation can ignore the BOM and still read everything as SHIFT-JIS."
 - Hubert: "If you did that, the BOM is not a BOM..."
- Jens suggested that the wording needs to establish when encoding determination happens; that should be the first step of translation phase 1.
- Jens added that the wording should be consistent with regard to encoding vs encoding form vs encoding scheme.
- Tom stated that, for UTF-8, encoding form vs encoding scheme doesn't matter, but that encoding scheme should be used if the intent is for the wording to be compatible with UTF-16 or UTF-32.
- Hubert asserted that, since the context is byte oriented files, encoding scheme should be used.
- Jens reiterated the necessary wording updates; the encoding scheme to use must first be established, then the source file can be validated and diagnostics issued if it fails to conform to the encoding scheme.
- Jens added that the wording needs to prevent the current implementation-defined mapping to the internal encoding from being applied to UTF-8 source files.
- PBindels asked if the added sentence in translation phase 2 regarding the "first codepoint" applies to each source file or just to the primary source file.
- Tom and Corentin replied that translation phases 1 through 3 are performed separately for each source file.
- Hubert suggested that translation phase 2 should discard a lead U+FEFF character regardless of the source file encoding.
- Jens noted that the added translation phase 2 sentence doesn't make sense without the wording changes proposed in [P2314: Character sets and encodings](#) due to character translation to *universal-character-name* in translation phase 1.
- Tom noted that the wording changes in P2314 allow distinguishing a source file with a BOM and a source file that starts with a `\uFEFF universal-character-name`.
- Jens clarified that, after P2314, a *universal-character-name* isn't translated to a UCS scalar value until translation phase 3.

- Hubert stated that it is a design question whether we want to treat a leading `\uFEFF` *universal-character-name* as a BOM.
- PBrett asked PBindels if he is satisfied with the BOM design following prior discussion.
- PBindels responded that he is, so long as we don't intentionally or unintentionally create the situation where UTF-8 source files end up requiring a BOM in practice.
- PBrett asked if we should add normative encouragement not to require a BOM.
- Hubert noted that, as wording updates are done, care must be taken to ensure we don't lose the wording that requires an implementation to accept a UTF-8 encoded source file whether it does, or does not, contain a BOM.
- Tom asked about handling of differently encoded source files.
- JeanHeyd replied in chat, "I think it's better to leave Encoding Identification to Tom's Paper on the subject."
- Tom replied in chat, "Assuming I actually deliver on that threat..."
- Hubert responded that the implementation must provide some means for standard headers (as opposed to header files), to remain usable when the implementation is running in UTF-8 mode.
- Steve added in chat, "Which might be 7 bit ascii for those headers. Which is largely the case today."
- **We wish to require implementations to support UTF-8 source files.**
 - **Attendance: 10**
 - **No objections to unanimous consent.**
- **We wish to require implementations to be capable of accepting UTF-8 source files whether or not they begin with a U+FEFF byte order mark.**
 - **Attendance: 10**
 - **No objections to unanimous consent.**
- Hubert reported that Clang allows non-UTF-8 encoded header names in `#include` directives in otherwise UTF-8 encoded source files.
- Steve stated that, since file names are not required to be representable in UTF-8, requiring strictly well-formed UTF-8 could have unanticipated consequences.
- JeanHeyd asked in chat, "Does `\xFF` work in header-names as an escape?"
- Corentin replied in chat, "unspecified".
- Corentin explained his intent in requiring diagnosis of ill-formed UTF-8 input.
- PBindels asked why it is useful to allow invalid UTF-8 in comments.
- Corentin replied that Clang source code has comments explaining why invalid UTF-8 in comments is explicitly allowed and provided a link to the source code.
 - <https://github.com/llvm/llvm-project/blob/main/clang/lib/Lex/Lexer.cpp#L3136-L3144>
- PBrett shared cases of copyright symbols appearing in otherwise ASCII files.
- Tom noted that non-ASCII characters tend to appear in author, product, and company names in comments.
- Hubert stated that source files that `iconv` will reject are undesirable.

- **We wish to require implementations to have a mode in which they diagnose ill-formed UTF-8 source files (regardless of whether the ill-formedness is located in comments, header names or string literals).**
 - Attendance: 10
 - SF F N A SA
8 2 0 0 0
- **Consensus is strongly in favor.**
- **SF: As it stands right now, people are already basically rolling the dice with their source files. This is strictly an improvement over the status quo, because now there is, at least, one entirely portable way to write source code.**
- Corentin asked about necessary wording to support both source files and non-files.
- Hubert responded that (standard library) headers are not source files; source files are those things that are included by `#include` directives that do not name standard headers.
- PBrett asked if the wording should be modified to discuss "input" as opposed to "files".
- Hubert responded that such a change is not necessary.
- Corentin pledged to bring back a revised paper.
- Tom stated the next telecon will be April 28th.

April 28th, 2021

Draft agenda:

- [LWG3547: Time formatters should not be locale sensitive by default](#)
- [P2093R5: Formatted output](#)
- [P2295R3: Support for UTF-8 as a portable source file encoding](#)
- [P2348R0: Whitespaces Wording Revamp](#)

Attendees:

- Charlie Barto
- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

- Zach Laine

Meeting summary:

- Charlie Barto was welcomed with a round of introductions.
- PBrett introduced the agenda.
- [LWG3547: Time formatters should not be locale sensitive by default](#)
 - PBrett presented:
 - Peter's presentation slides are available [here](#).
 - As currently specified, whether a format specifier is locale dependent is not obvious.
 - Floating point values are locale independent by default, but chrono values are not.
 - There is no systematic way to format locale-independent and locale-dependent chrono values.
 - Victor expressed a preference for chrono values being locale independent by default.
 - Victor explained that the current specification derived from existing specifiers used elsewhere.
 - Victor noted that, in some cases, specifiers are not available for locale independent formatting.
 - Victor reported success with a prototype implementation of the proposed resolution that performs locale independent formatting of chrono values unless a L specifier is present.
 - Charlie stated that changes to the format specifier syntax may have more implementation impact than just requiring changes to the implementation behavior.
 - *[Editor's note: Discussion regarding the amount of time available to make changes before implementations of `std::format()` are shipped to users ensued. That discussion is not recorded as it involved discussion of internal company time lines that have not yet been stated in public.]*
 - PBrett noted that there are two related issues:
 - 1: The format specification syntax.
 - 2: The behavior of the format specifiers.
 - PBrett explained that the proposed resolution addresses both concerns by making the format syntax consistent in requiring a L specifier to opt-in to locale dependent behavior.
 - Charlie noted that `std::format()` does not currently perform any transcoding operations today; not for format arguments, and not for text provided by a locale that uses a different character encoding than the literal encoding.
 - Charlie added that `std::format()` does need to be encoding aware for the purposes of field width estimation.
 - Corentin stated that the intent of the proposed resolution is to ensure that `std::format()` use consistent syntax to opt-in to locale dependent formatting and

encouraged trying to address at least this concern.

- Corentin added that LWG might agree on a resolution in a short time frame, but that there will not be a plenary poll until June.
- PBrett stated that the resolution may be considered evolutionary.
- Victor agreed and noted that the `L` specifier could be added for a future standard.
- Victor asserted that we do need to decide what the default behavior is now.
- Victor added that we could consider transcoding locale provided text and potentially detecting mojibake if it would be produced.
- Victor noted that the format string is always a literal.
- *[Editor's note: In C++20, the format string may not be a literal, but [P2216](#), if adopted, will require a literal or other compile-time evaluated expression.]*
- Zach asked for clarification regarding what is meant by "default behavior" and noted that the `%0u` specifier is locale dependent, but that `%u` is not.
- Victor responded that there are cases like `%T` that do not have locale independent forms.
- *[Editor's note: `%T` is locale dependent because the decimal point character potentially used for sub-second precision is provided by the locale.]*
- Hubert stated that these concerns will be difficult to resolve quickly, are clearly evolutionary, and may require balloting.
- Hubert added that there may also be issues with requiring the locale independent behavior to use English translations.
- Tom noted that the basic source character set already has a bias in English.
- Hubert responded that this goes further; we may potentially have to specify behavior in terms of `asctime()`.
- Charlie commented that the text provided by the locale facet is currently produced by the operating system; changing that behavior may not be problematic.
- Charlie added that adding new format specifiers will result in incompatibilities if code that uses those specifiers is run with an older library implementation that doesn't support them.
- Charlie noted that, if support for compile-time format string checking is adopted via [P2216](#), then the format string will become part of the function template specialization; this may help to avoid library compatibility issues.
- Charlie stated that there are multiple sources of locale information and that formatting of the chrono types is governed by the Windows region settings.
- Charlie noted that changes to the Windows region settings require a reboot.
- Tom asked for confirmation that calls to `std::setlocale()` don't affect how chrono values are formatted.
- Charlie confirmed that is correct.
- PBrett asked if `std::format()` behavior is affected by changes to the global locale via `std::locale::global()`.
- Charlie responded that the global locale does affect the behavior of format specifiers that include the `L` specifier.
- Charlie clarified that the global locale will not affect parsing of the format string itself.

- Corentin requested review of the proposed resolution.
- Hubert noted that the wording requires that the "C" locale be used for field formats that do not include the `L` specifier regardless of whether a `std::locale` argument is passed.
- Hubert noted that under the C++20 wording, implementations trying to accomodate this tentative future direction may be more able to ignore the global locale than an explicit locale argument. So, a change that maintains respecting the locale parameter is more compatible with C++20.
- Tom responded that doing so would not be consistent with the other standard format specifiers.
- Victor agreed and added that he would be strongly opposed to implicit use of a `std::locale` parameter.
- Jens stated that a migration path to better behavior needs to be established and noted that the current situation is an interesting mess.
- Jens suggested investigating how to increase consistency with the existing locale dependent format specifiers; e.g., for decimal point and digit group separator characters.
- Jens added that there may be cases where it would be useful to be able to specify use of the "C" locale even when a locale is provided as an argument.
- Jens observed that use of the "C" locale for the `chrono %p` specifier would be consistent with use of the "C" locale for floating point values.
- Jens noted that the example in the proposed resolution does not match the proposed grammar; the `L` specifier should precede the *chrono-specs* specifier, not follow it.
- Jens stated that adding support for the `L` specifier is backward compatible from a standard evolution perspective.
- Tom stated that a change to use the "C" locale in place of the global locale or a locale passed as an argument can be done as a non-abi breaking change.
- Charlie agreed, but noted that some implementation tricks may be required to avoid potential conflicts with older libraries.
- Zach stated that mixing different library versions is non-conforming anyway.
- Corentin stated that the "C" locale is used as a proxy for the absence of a locale and suggested that a `constexpr locale` might be desired in the future.
- Corentin asked Charlie if formatters can be modified without breaking ABI.
- Charlie replied that they are templates, so modifications can result in ODR violations. Charlie added that inline namespaces can be helpful in some cases.
- PBrett asked for confirmation that use of a `L` specifier where one is not expected will result in a format exception being thrown.
- Victor confirmed that is the case.
- PBrett asked if the `L` specifier could be reserved now such that a format exception will be thrown if used, and then different behavior specified later.
- Charlie responded that changing behavior to not throw in cases where an exception was previously thrown is fine so long as mixed library version problems are avoided.

- Victor expressed agreement with Jens' prior comments.
- Victor stated that behavior must remain consistent between `std::format()` overloads that do and do not accept `std::locale` arguments; the presence of the `std::locale` argument must not, by itself, affect behavior.
- PBrett suggested that a paper that explores the alternatives may be required.
- Corentin asserted that it must be possible to evolve the `std::format` format string so as to add new behaviors.
- Corentin expressed distaste for the idea of a "no locale" specifier; that approach would still result in inconsistencies with number formatting.
- Charlie agreed.
- Jens conceded that challenging standardization work will be required if behavior changes from C++20 to C++23.
- Jens asserted that the right to add format specifiers when a new standard is issued must be reserved, even if doing so causes implementation challenges.
- **Poll 1: LWG3547 raises a valid design defect in [time.format] in C++20.**
 - **Attendance: 11**
 - **SF F N A SA**
7 2 2 0 0
 - **Consensus: Strong consensus that this issue represents a design defect.**
- Hubert noted that, with regard to issues of consistency, the proposed resolution is a departure from existing interfaces such as `strftime()`.
- **Poll 2: The proposed LWG3547 resolution as written should be applied to C++23.**
 - **Attendance: 11**
 - **SF F N A SA**
0 4 2 4 1
 - **No consensus.**
 - **SA: Mitigation of behavior changes sensitive to string literal contents is very difficult and there are options available to deal with this problem in an additive way; this direction represents an unnecessary backward compatibility break.**
- Mark stated that the proposed resolution would have been great 18 months ago.
- PBrett responded that we need to recognize when we make mistakes and own correcting them.
- Corentin lamented the current state being another case of a bad default.
- Tom suggested that the current behavior can be presented as intentional with the goal to maintain consistency with existing interfaces; new format specifiers can then be added in C++23.
- PBrett suggested that an SG16 issue be filed and a volunteer found to work on it.
- Victor responded that the behavior isn't sufficiently broken to make him want to spend time on it.
- *[Editor's note: Despite that lack of desire, Victor and Corentin quickly authored an initial draft paper that will become [P2372R0](#) once published.]*

- PBrett volunteered to work on a paper.
- Tom and PBrett thanked Charlie for joining the telecon and encouraged him to continue attending.
- Tom stated that Victor had expressed interest in working on a potential `std::locale` replacement and asked if there were other volunteers interested in such work.
 - Victor responded that the motivation was provided by Hubert's example code included in the telecon agenda, that he is interested in conducting some implementation experiments, but that he does not have anything concrete in mind yet.
 - *[Editor's note: Hubert's example is below. In addition to the question of which locale is used in the formatting, there is a question of how encoding issues are handled. The example depends on a locale to provide translations of AM/PM designators for a 12-hour clock. What happens when the literal encoding is UTF-8 and the locale provides translations in Windows codepage 932?]*

```
std::print("{:%r}\n", std::chrono::system_clock::now().time_since_epoch());
```
 - PBrett expressed interest in being involved.
- Tom stated that the next SG16 telecon will be held May 12th.
 - Tom added that the agenda will include further discussion of [P2093R5: Formatted output](#) and a return to [P2295R3: Support for UTF-8 as a portable source file encoding](#).
 - PBrett asked if a CWG expert could review and comment on the updated wording for P2295R3.
 - Hubert agreed to do so.
 - Corentin requested a CWG expert also review the proposed wording in [P2348R0: Whitespaces Wording Revamp](#).

May 12th, 2021

Draft agenda:

- [P2295R3: Support for UTF-8 as a portable source file encoding](#)
- [P2372R1: Fixing locale handling in chrono formatters](#)
- [P2093R6: Formatted output](#)
- [P2348R0: Whitespaces Wording Revamp](#)

Attendees:

- Charlie Barto
- Hubert Tong
- Jens Maurer

- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2295R3: Support for UTF-8 as a portable source file encoding](#)
 - No discussion as the author was not present.
- [P2372R1: Fixing locale handling in chrono formatters](#)
 - *[Editor's note: D2372R1 was the active paper under discussion at the telecon. That paper was later published as P2372R1 without further modification. The agenda and links used here reference P2372R1 since the links to the draft paper were ephemeral.]*
 - PBrett introduced the topic:
 - LEWG reached consensus for the direction proposed by [P2372R0](#) at its [2021-05-03 telecon](#) with additional refinement to preserve locale dependent formatting for iostreams.
 - Since SG16 polls conducted at its [2021-04-28 telecon](#) did not agree with this direction, LEWG requested that SG16 review and conform or rebut the LEWG consensus.
 - Victor presented slides lightly updated from his prior LEWG presentation.
 - Victor's presentation slides are available [here](#).
 - **Poll 1: Forward D2372R1 to LEWG for inclusion in C++23 and with the intent that it be applied retroactively to C++20.**
 - **Attendance: 8**
 - **SF F N A SA**
5 2 1 0 0
 - **Consensus: Strong consensus in favor.**
 - *[Editor's note: D2372R1 contains the LEWG requested update to preserve locale dependent formatting for ostreams.]*
 - *[Editor's note: The chair's perception is that SG16's change in consensus is attributable to two factors:*
 - 1. New information that arrived after the initial poll.*
 - 2. SG16's original poll targeted C++23 while LEWG's poll targets C++23 and C++20 as a DR; some concerns had been expressed regarding backward compatibility and migration.*
 - *]*
- [P2093R6: Formatted output](#)
 - Victor presented:
 - `std::print()` integrates `std::format()` with I/O.

- R6 addresses recent LEWG feedback:
 - The proposed `std::print()` header was changed from `<io>` to `<print>`.
 - Additional rationale and clarifications were added regarding:
 - Substitution of replacement characters.
 - The choice to base behavior on the compile-time literal encoding.
 - ANSI escape sequences do not constitute a native device API.
 - Existing practice in Rust.
- PBrett asked how substitutions would be performed for different kinds of ill-formed scenarios.
- Zach stated that the Unicode standard documents recommended practice for substitution of replacement characters.
- *[Editor's note: [Unicode 13](#) discusses substitution of replacement characters in section "U+FFFD Substitution of Maximal Subparts" of chapter 3.9, "Unicode Encoding Forms" and in chapter 5.22, "U+FFFD Substitution in Conversion".]*
- Zach expressed a preference for implementations to be consistent in how replacement characters are substituted.
- Hubert stated that an example should be added to the paper.
- Hubert expressed a preference for `vprint_unicode()` to substitute replacement characters even when the output device is not Unicode.
- Victor asked if that could be done as implementation-defined behavior.
- Hubert responded, no; the goal is for the substitution behavior to be deterministic for `vprint_unicode()` regardless of the output device.
- Victor replied that he would prefer that behavior to be optional.
- Hubert replied that he would like to ensure that ill-formed inputs are not presented with no indication that something went wrong.
- PBrett stated that, when writing to a Unicode device, a U+FFFD replacement character should be substituted and the device should then handle it as its designers intended.
- Victor agreed with the substitution rationale for the device case since transcoding may be necessary, but disagreed for files due to a desire to avoid the validation overhead.
- Hubert expressed a preference for the behavior of `vprint_unicode()` to be consistent across files and devices.
- PBrett suggested that what Hubert desires is some kind of noisy failure, like a trap.
- Hubert agreed and restated the goal as some kind of signal that encoding issues were encountered.
- Steve stated that C++ programs do not typically interact directly with a device and that it is difficult to diagnose problems where the data can't be inspected en route.
- PBrett asked if Steve had a suggestion.

- Steve responded with a preference for a programatic error handling facility.
- Zach stated that, in the case where UTF-8 source is copied to a UTF-8 sink, introduction of replacement characters could be surprising, but when transcoding is required, e.g., when the sink is UTF-16, then replacement characters are expected.
- Zach suggested decomposing the problem; validate and handle errors first, then convert.
- Charlie explained that, on Windows, the only ways to write Unicode to the console are to change the console encoding and write using the ANSI APIs, or to convert to UTF-16 and write using the wide APIs.
- Charlie noted that, since the console encoding is a global property of the process, changing it within `std::print()` would require synchronization.
- Zach suggested that it is reasonable to get mojibake in the ANSI case if the console encoding hasn't been correctly set.
- Hubert responded that the global console encoding condition seems to be particular to Windows and worth addressing.
- Charlie pondered the ramifications of writing to a stream opened in text mode.
- Victor reiterated his stance on not wanting to pay validation costs except in cases where transcoding is necessitated.
- **Poll 2: When `<print>` facilities must transcode formatting results for display on a device and, during that process, invalidly-encoded text is encountered, `std::print()` should replace the erroneously-encoded code units with U+FFFD REPLACEMENT CHARACTER.**
 - Attendance: 9
 - SF F N A SA
3 3 1 2 0
 - Consensus is in favor.
 - A: Not convinced that silently substituting replacement characters is always the right policy; an exception could be appropriate. There are parallels with integer overflow.
 - A: Testing is difficult if substitution is device sensitive.
- Charlie expressed support for a direction that would allow explicitly inhibiting use of the native device API but noted that, on Windows, that would mean the console encoding would have to be correctly set and the application would have to take care of buffering concerns.
- **Poll 3: When `<print>` facilities need not transcode their formatting results for display on a device and invalidly-encoded text is encountered, `std::print()` should nevertheless replace the erroneously-encoded code units with U+FFFD REPLACEMENT CHARACTER.**
 - Attendance: 9
 - SF F N A SA
1 0 2 2 3
 - N: Undecided due to uncertainty; more consideration is needed.

- **A:** Would prefer a UB approach that would enable sanitizers to diagnose these cases and remain conforming.
- **SA:** There is lack of implementation experience for this direction, it imposes overhead, and there are terminals that accept bytes.
- **SA:** A wide contract with validation does not make sense for high-performance I/O.
- PBrett stated that there appear to be different audiences for `std::print()` and these audiences have different ideas of what is "obviously" correct:
 - For some, `std::print()` is a simple tool that enables a better Hello World.
 - For others, it is a high-performance I/O facility.
 - For yet others, it is a way to format bytes.
- Tom suggested that an error handling facility might move us towards more consensus.
- PBrett noted that something like JeanHeyd's transcoding facilities could provide that.
- Charlie agreed that integration of a familiar transcoding facility could work.
- Tom stated that the next telecon will be May 26th and that the agenda will again include [P2295R3](#) and [P2093R6](#).

May 26th, 2021

Draft agenda:

- [P2295R4: Support for UTF-8 as a portable source file encoding](#)
 - Review updates intended to address prior SG16 feedback.
- [P2093R6: Formatted output](#)
 - Discuss locale dependent character encoding concerns.

Attendees:

- Corentin Jabot
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2295R4: Support for UTF-8 as a portable source file encoding](#)
 - *[Editor's note: D2295R4 was the active paper under discussion at the telecon. The agenda and links used here reference P2295R4 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - PBrett provided an introduction.
 - Corentin presented and described the changes from R3 to the draft R4.
 - PBrett observed that the wording updates removed the prior definition for a *UTF-8 file* and added a new definition for a *UTF-8 source file*.
 - Tom recalled prior discussion that suggested there was no need to provide such a definition at all.
 - Jens confirmed and explained that the prior suggestion was to instead specify translation phase 1 in terms of a sequence of characters instead.
 - Jens noted that there will be merge conflicts with [P2314](#).
 - Corentin asked if the merge conflicts can be dealt with after CWG reviews P2314.
 - Jens confirmed that they can be.
 - PBrett asked if progress can be made before P2314 is adopted into the working paper.
 - Jens confirmed that progress can be made.
 - PBrett asked Jens if he would like to see additional wording changes reviewed in SG16.
 - Jens replied that he would and noted that he had not received a response to all of the suggestions previously provided in his message to the mailing list available at <https://lists.isocpp.org/sg16/2021/04/2353.php>.
 - Jens observed that the proposed wording results in existing wording no longer applying to all source files. For example, "Any source file character not in the basic source character set is replaced by the *universal-character-name* that designates that character" now appears in a paragraph that doesn't apply to UTF-8 source files.
 - Corentin responded that this paper doesn't make sense without the changes from P2314.
 - Tom asked if the wording could be rebased on P2314 with a noted dependency on P2314.
 - Jens replied that it could be.
 - Hubert noted that the definition of a UTF-8 source file is problematic since the definition could apply to a file that just so happens to decode as UTF-8, but is not intended as a UTF-8 file.
 - PBrett responded that the following sentence specifies that encoding determination is implementation-defined.
 - Hubert acknowledged and suggested it might be helpful to reorder the sentences.

- Hubert added that wording is still required to reflect intent that a file be interpreted as UTF-8.
- PBrett agreed by way of an example; an implementation invoked without such intent may analyze a file, determine that it does not decode successfully as UTF-8, and then interpret it as, for example, Windows-1252, and do so without issuing a diagnostic.
- Jens observed that the wording states that, "An implementation shall support UTF-8 source files", but there is no wording to require diagnosis of ill-formed UTF-8 source files.
- Corentin responded that there is no such thing as an invalid UTF-8 file; either a file is valid UTF-8 or it is not UTF-8.
- Mark responded that there is a desire to have implementations produce a diagnostic if source files that are purported to be encoded as UTF-8 are not, in fact, valid UTF-8.
- PBrett stated that there are three distinct requirements:
 - A requirement to support UTF-8 encoded source files.
 - A requirement for means to inform the implementation that all source files are to be assumed to be UTF-8 encoded.
 - A requirement that the implementation diagnose files that were assumed to be UTF-8 encoded but that contain (some) non-UTF-8 content.
- Hubert offered some suggested wording in chat:
 - "An implementation shall provide for processing physical source files as having a UTF-8 encoding scheme without restriction, other than resource limits ([implimits]), upon the content of the physical source file."
- Jens pasted previously suggested wording from the mailing list in chat:
 - "The encoding scheme of a physical source file is determined in an implementation-defined manner. An implementation shall support (possibly among others) the UTF-8 encoding scheme."
 - "If the encoding scheme of a physical source file is determined to be UTF-8, the physical source file shall consist of a well-formed sequence of UTF-8 code units as specified by ISO/IEC 10646."
- Hubert expressed support for that wording but thought some additional updates would still be required to ensure diagnostics.
- Corentin disagreed with removal of wording that requires that the scalar value of source file characters be preserved.
- Jens responded that the scalar value preservation wording isn't required because the mapping to the translation character set already preserves characters.
- Steve noted the existence of wording that uses the phrase "known to the implementation" and asked if that could be used to specify how source file encoding is determined.
- Tom suggested that implementation-defined is preferred since that reflects a documentation requirement.

- Hubert added that the "known to the implementation" wording is not intended to reflect that implementations can be wrong.
- PBrett observed that Jens and Hubert would presumably like to see updated wording.
- Hubert expressed a belief that the required wording has been identified and that he is onboard with the goal of preserving scalar value sequences from UTF-8 source files.
- Corentin responded that he will bring back a revised paper with the suggested wording.
- Steve informed the group that the EWG chair is considering dedicating a telecon to SG16 papers in the next month or so.
- [P2093R6: Formatted output](#)
 - PBrett reported a previous conversation with Victor in which Victor expressed that he felt he has the guidance he needs regarding handling of substitution characters and locale.
 - Victor presented slides:
 - The next question to be answered is whether it is ok to base behavior on the literal encoding.
 - Use of the literal encoding avoids race conditions with locale settings.
 - Discussion ensued regarding current dependencies on the choice of literal encoding and it was observed that, though the wording provided by [P1868](#) to specify estimated format field widths is not based on the literal encoding, at least one implementation is planning to only use the specified estimated widths when the literal encoding is UTF-8.
 - Hubert observed that field width estimation can apply to content from other than string literals.
 - PBrett provided an example; when `gettext()` is used, a literal is used for the message catalog lookup, but the result is not a string literal.
 - Hubert acknowledged the provided rationale, but noted that it does not address concerns raised and that he has seen many cases where use of locales works fine on UNIX systems.
 - Hubert added that this has the potential to bite existing users since code may appear to work correctly until it suddenly doesn't.
 - Victor replied that his goal is to make UTF-8 cases work as expected and that he is willing to accept some surprises in other scenarios.
 - Victor stressed that the intention is that, on UNIX systems, bytes are simply passed through.
 - Tom directed discussion towards the example code from the [telecon announcement](#).
 - Victor stated that he will request a LWG issue or author a paper to address handling of locale provided text.
 - *[Editor's note: Victor requested an LWG issue that is now tracked as [LWG issue 3565](#).]*

- Corentin stated that he is content with undefined behavior for cases where UTF-8 input is expected, but the input is not actually UTF-8 encoded.
- Hubert responded that the format locale situation is rather urgent for EBCDIC environments.
- PBrett stated that he is ok with the proposal because it won't break anything worse than it already is.
- Tom stated that the next telecon will be held on June 9th.